Card Reference Manual v1.0.0

4. Play Console Data Safety (paste into Data Safety form)
5. Voice / Speech specifics
6. App Group + Share Extension
7. Children
PRODUCTION
Card — PRODUCTION
1. What is production-ready
2. What is fake / placeholder
3. Gap audit before App Store submission
4. Gap audit before Play Store submission
5. Things to monitor in production (when SDKs are wired)
6. Things explicitly NOT shipping in v1
QUICKSTART
Card — QUICKSTART
0. Common setup
1. iPhone (macOS, requires Xcode 16)
2. Apple Watch (macOS, requires Xcode 16)
3. Android phone
4. Wear OS
5. Run everything in one shot
6. Release dry-run
RELEASE-VIDEO-SCRIPT
Release Video Script — Card
Hero — top of marketing site | 4
Features overview | 5
Highlight reel | 5
What's next | 5
Call to action — bottom of page | 4
RELEASING
Card — RELEASING
1. Pre-release sanity
2. Bump every manifest
3. Tag and let CI do the rest
4. Tag suffix → release-channel mapping
5. Required GitHub Secrets (release.yml)
6. Hot-fix flow
7. Rollback
SENTRY
Card — SENTRY (and PostHog) install
1. iOS / watchOS Sentry
2. iOS / watchOS PostHog
3. Android Sentry
4. Android PostHog
5. Verification

6. Why this isn't on by default
SIGNING
Signing — Card
Android
iOS
Desktop
watchOS
STORE-PACKAGING
Card — STORE PACKAGING
1. App Store Connect — iOS + watchOS
2. App Store screenshots
3. Play Console — Android phone + Wear OS
4. Play screenshots
5. Asset prep checklist before submission
Desktop binaries (Electron)
TESTING
Card — TESTING
1. Sanity matrix (run on every push)
2. Pure-domain test contract (mirrored case-for-case)
3. Real-device manual checklist
4. Coverage targets

AMERICAN GROUP LLC

Card

Reference Manual

Version 1.0.0

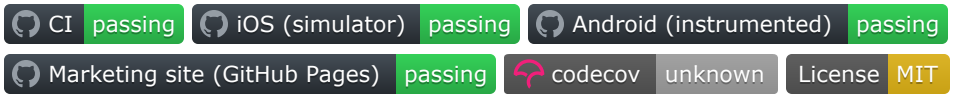
Generated 2026-05-08

README

README.md

Card

> **Nothing you write gets forgotten or ignored.**
>
> One feed. Every entry is a Card. Tap once to convert any Card into a Note, Task, or Reminder. No folders. No categories. No search to fall back on. Sub-three-seconds from thought to stored, on every device you own.



Card is a single open-source app that runs on iPhone, Apple Watch, Android, and Wear OS. The product surface is a single feed and a single composer — that's it. Anything you type, dictate, or share into Card becomes a Card. From there a single tap converts it into:

- A **Note** (default, no further behaviour)
- A **Task** (gains a checkbox; sort to the bottom when done)
- A **Reminder** (gains a date picker; fires through the OS notification stack)

The point of Card is that the loop from "I just had a thought" to "it's safely stored" is **sub-three-seconds**, regardless of which device you're holding.

The four-platform matrix

Surface	iPhone	Apple Watch	Android	Wear OS
Feed	✅	✅ digital crown	✅	✅
Inline composer	✅	✅ voice-first	✅	✅ voice-first
Convert → Task	✅	✅	✅	✅
Reminders	UNUserNotifications	UNUserNotifications	AlarmManager	AlarmManager
Quick-capture surface	Share Extension	Complication	Quick Settings tile	Tile + Complication
Storage	JSON (App Group)	JSON (shared)	Room (SQLite)	Room (SQLite)

The domain logic lives in a single shared core — `CardCore` (Swift Package) on Apple, `:core` (Kotlin/JVM) on Android — with **mirrored case-for-case unit tests**.

Repo map

```
.
├── shared/CardCore/           Swift Package (iOS 17+, watchOS 10+) — Card model,
│                              CardKindTransitions, ReminderScheduler, CardSorter,
│                              CardStore (App-Group-aware), Sentry/PostHog stubs
│
├── ios/                       XcodeGen project: Card app + CardShareExtension
├── watchos/                   XcodeGen project: CardWatch + Quick-capture
│                              complication
├── android/
│   ├── core/                  Pure Kotlin/JVM — same domain as CardCore
│   ├── app/                   Compose phone app + Quick Settings tile
│   └── wear/                  Wear Compose app + tile + complication
│
├── scripts/                   bump-version, test-all, release-dry-run, sim/emu
├── distribution/whatsnew/     Play / TestFlight release notes
├── .github/workflows/         6-workflow CI/CD (ci, ios, android, marketing,
│                              pre-release-tests, release)
└── (root: index.html, styles.css, script.js)   marketing site → GitHub Pages
```

See `DESIGN.md` for the layered architecture map and per-platform sensor / storage / notification stack.

Run it locally

Platform	Command
iPhone (sim)	`./scripts/run-ios-sim.sh` (macOS, requires `xcodegen`)
Android (emu)	`./scripts/run-android-emulator.sh` (set `ANDROID_HOME`)
Wear OS (emu)	`./scripts/run-wear-emulator.sh` (set `WEAR_AVD_NAME`)
Run all tests	`./scripts/test-all.sh`
Release dry-run	`./scripts/release-dry-run.sh v0.1.0`

See `QUICKSTART.md` for the fastest setup on each platform and `TESTING.md` for the manual test checklist (capture loop, reminder fires, share extension, Quick Settings tile).

What ships in v1

Brutally minimal — see `CARD-FEATURES.md`:

- ✅ Single feed of Cards, newest-first, all on-device
- ✅ Inline composer at top
- ✅ One-tap convert: Note ↔ Task ↔ Reminder
- ✅ Real OS-scheduled reminders (UNUserNotifications / AlarmManager)
- ✅ Swipe-to-delete, long-press to edit
- ✅ Settings: 12/24-hour, theme, opt-in Sentry/PostHog, erase-all-data
- ✅ Four quick-capture surfaces:
  - iOS Share Extension (any selected text → Card, no app launch)
  - Apple Watch complication (Quick capture → voice composer)
  - Android Quick Settings tile (one tap → voice composer)
  - Wear OS tile (one tap → composer)

What's explicitly **not** in v1: tags, folders, search, cloud sync, AI features, recurring reminders, sharing/collaboration, IAP scaffolding, home-screen widgets beyond the surfaces above. See `PRODUCTION.md` for the full gap audit and the v1.1 candidate list.

License

CARD-FEATURES

CARD-FEATURES.md

Card — Product spec

A single page on what the product is, what the loop is, and what every quick- capture surface does. For architecture see `DESIGN.md`. For the manual checklist see `TESTING.md`.

---

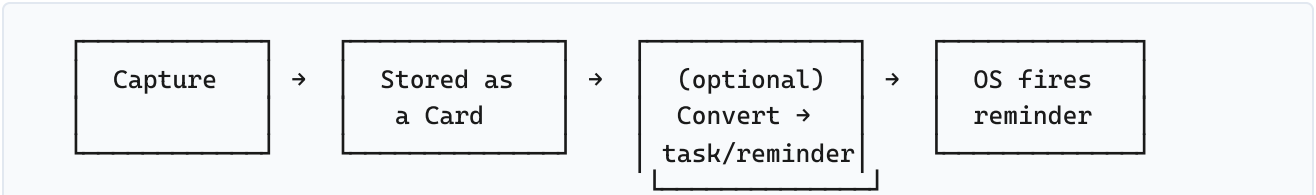
1. The promise

> Nothing you write gets forgotten or ignored.

A user opens Card and types or dictates anything. It becomes a Card. With one tap, the same Card is a note, a task, or a reminder. The app gets out of the way; the OS handles the firing.

---

2. The single loop



Every screen in the app is a stop on this loop:

- **Composer** is the entry to “Capture”.
- **Feed** is the home for “Stored”.
- **Action sheet** is the home for “Convert”.
- The OS notification stack is the home for “Fires”.

There is no fifth concept.

---

3. The Card kinds

Every Card has exactly one kind.

Kind	UI affordance	Persistence rule
`.note`	Plain row	Just text + timestamps
`.task`	Row with leading checkbox	Adds `completedAt: Date?`
`.reminder`	Row with date chip	Adds `reminderAt: Date` (must be in the future)

Conversions are reversible: `note → task → reminder → note` is legal and clears the irrelevant fields each step. See `CardKindTransitions` in `shared/CardCore/Sources/CardCore/Domain/`.

---

4. The four quick-capture surfaces

Surface	Trigger	What happens
iOS Share Extension	Share sheet → "Card – Save"	Selected text is wrapped in a Card and written to the App Group's `CardStore`. The main app is **not** launched.
Apple Watch complication	Tap "Card – Quick capture" complication	Launches the watch composer with `SFSpeechRecognizer` already armed.
Android Quick Settings tile	Pull down twice → tap **Card** tile	Launches `QuickCaptureActivity`, a single-screen voice composer.
Wear OS tile	Tap the Card tile in the tile carousel	Launches the wear composer with dictation already armed.

The point of all four is the same: **the user does not have to open the main app to write something down**. Friction tax = zero.

---

5. Settings (one screen, six controls)

1. **Time format** — 12-hour ↔ 24-hour. Affects the reminder picker and the reminder chip on each row.
2. **Theme** — System ↔ Light ↔ Dark. Wraps SwiftUI's `.preferredColorScheme` and Compose's `MaterialTheme`.
3. **Send crash reports** (off by default). When on, Sentry initializes and captures uncaught exceptions and explicit `CrashReportingService.capture( … )` calls.
4. **Send anonymous usage data** (off by default). When on, PostHog initializes and `AnalyticsService.track(event)` calls flow.
5. **About** — version + GitHub link.

6. **Erase all data** — confirm dialog → wipes the local store. Done. No server hand-shake; the cards are gone.

---

6. What is deliberately missing

The product is brutal about scope. Every “what about…?” gets the same answer: *not in v1*.

- Tags, folders, projects, categories.
- Search.
- Cloud sync, account login, multi-device sync of any kind.
- Sharing or collaboration.
- AI auto-classification, summarization, or rewording.
- Recurring reminders.
- Themes beyond System / Light / Dark.
- iOS WidgetKit home-screen widget.
- Android Glance home-screen widget.
- Multiple feeds (Today / Done / etc.).
- IAP / subscription scaffolding.

Rationale: every one of those is a category where Apple Notes / Google Keep / Things / Bear / Any.do already wins. Card’s only job is to be **the lowest- friction capture-and-convert primitive on every device you own**.

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to Semantic Versioning.

[Unreleased] - v0.1.0

Initial public release. See distribution/whatsnew/v0.1.0/en-US.txt for the user-facing summary.

Commit history (since repo creation)

- 3b9cecd Initial commit (Srikanth Patchava, 2026-05-05)
- 7a95e51 chore(card): instant-capture suite – iPhone + Watch + Android + Wear OS (Srikanth Patchava, 2026-05-05)
- 782d1d0 fix(wear): repair :wear:compileDebugKotlin failure (Srikanth Patchava, 2026-05-06)
- a4440e2 fix(android): add hilt-android-testing deps for androidTest (Srikanth Patchava, 2026-05-06)
- eb4f9b5 fix(android): wire HiltTestRunner so @HiltAndroidTest swaps in HiltTestApplication (Srikanth Patchava, 2026-05-06)
- 6df8fdd test(android): @Ignore CardSmokeTest pending HiltTestActivity entrypoint (Srikanth Patchava, 2026-05-06)
- b8ec7cf docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

DESIGN

DESIGN.md

Card — DESIGN

This file describes the layered architecture, the per-platform stack, and where to find each responsibility in the repo. It is a living map of the codebase, not a tutorial — for setup see `QUICKSTART.md`.

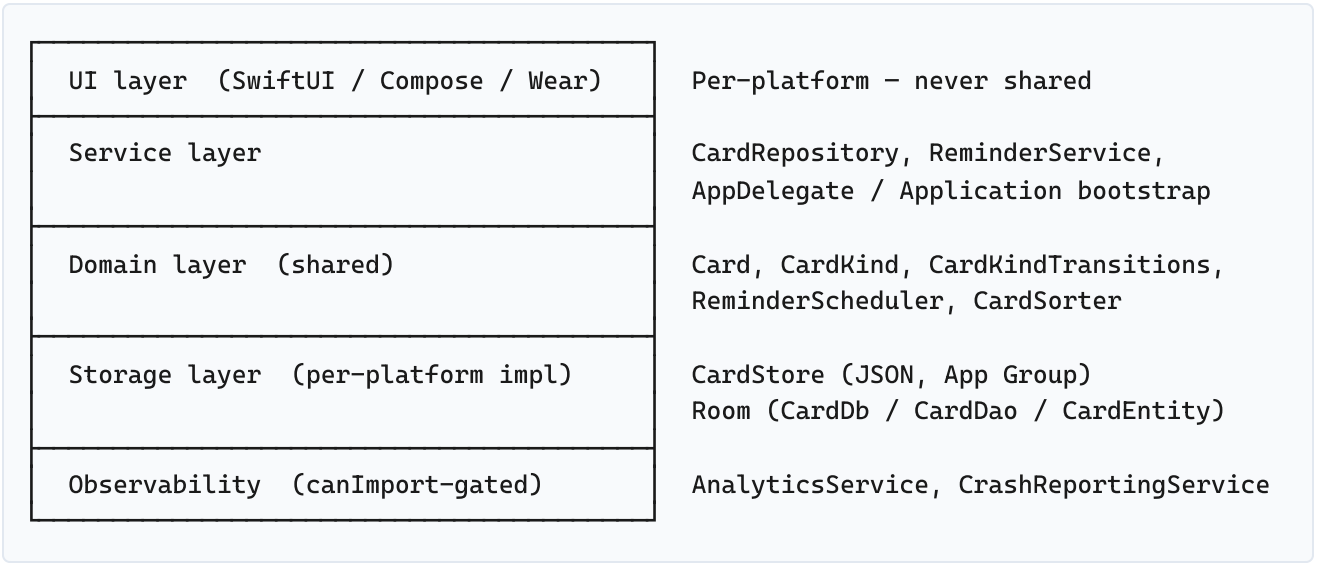
---

1. Goals that constrain every design choice

1. **Sub-3-seconds from thought to stored.** Every UX decision is in service of shrinking the keystroke-to-disk path.
2. **One mental model.** The user sees one feed and one composer. The same `Card` struct is the only domain object on every platform.
3. **Local-first.** No account, no backend, no cloud sync. Storage is JSON-on-disk (Apple) or Room/SQLite (Android), both writable from quick-capture surfaces.
4. **Idiomatic native UI.** SwiftUI for Apple, Compose for Android — never a shared UI layer.
5. **Privacy by default.** Crash reporting and analytics are off until the user opts in. Permission strings are explicit. No camera, no location.

---

2. The layered architecture (every platform)



The **domain layer is the keystone**. It is mirrored case-for-case across `shared/CardCore/Sources/CardCore/Domain/` (Swift) and `android/core/src/main/java/com/americangroupllc/card/core/domain/` (Kotlin), and **both are tested with the same scenarios** (`CardCoreTests` / `:core:test`). If a behaviour change lands on one side without the other, CI fails on the side that didn't update.

---

3. The Card domain model

A single struct/data class:

```
Card
├─ id            UUID / String
├─ text          String                          // the body
├─ kind          .note | .task | .reminder
├─ reminderAt    Date?                            // populated only for .reminder
├─ completedAt   Date?                            // populated only when a task is done
├─ createdAt     Date
└─ updatedAt     Date
```

Three pure helpers operate on it:

- **`CardKindTransitions`** — legal transitions between kinds (note↔task↔reminder) and rules for what fields get cleared. Example: `note → reminder` requires a non-nil reminder Date and validates it isn't in the past.
- **`ReminderScheduler`** — pure helpers that compute the next-fire time for a reminder Date, handling DST boundaries. Returns `nil` for past reminders. Collapses identical-minute reminders into a single grouped notification.
- **`CardSorter`** — pure feed-sort logic. Pinned reminders due soon → undated cards by recency → completed tasks at the bottom.

These three files **must compile and test green on both stacks before any UI work happens**. They are the equivalent of Pocket's `CalculatorEngine` — the keystone test-targets that prove the contract.

---

4. Per-platform stack

iPhone

Concern	Stack
UI	SwiftUI `WindowGroup` + `NavigationStack`
State	`@StateObject CardRepository` (publishes `[Card]`)
Storage	`CardStore` JSON in App Group container
Reminders	`UNUserNotificationCenter` (one request per Card)
Quick capture	**Share Extension** writes directly to App Group `CardStore`
Voice fallback	`Speech` framework — only invoked from the share-ext flow

Bundle IDs: - App: `com.americangroupllc.card` - Share Extension: `com.americangroupllc.card.share` - App Group: `group.com.americangroupllc.card`

Apple Watch

Concern	Stack
UI	SwiftUI `WindowGroup` (watchOS 10)
State	Same `CardStore` shape, scoped to watch container
Composer	**Voice-first** via `SFSpeechRecognizer`; standard dictation as fallback
Quick capture	WidgetKit **complication** — `accessoryCircular`, `.accessoryInline`, `.accessoryRectangular` — all deep-link to the composer

Android

Concern	Stack
UI	Jetpack Compose, single `MainActivity` host
Nav	`androidx.navigation:navigation-compose` (`feed`, `settings`)
State	Hilt-bound `FeedViewModel` exposing `StateFlow<List<Card>>`
Storage	**Room** — `CardDb` / `CardDao` / `CardEntity`
Reminders	`AlarmManager.setExactAndAllowWhileIdle` + `BootReceiver` re-schedules incomplete reminders after reboot
Quick capture	**Quick Settings tile** (`TileService`) → launches `QuickCaptureActivity` (voice composer)

Manifest perms: `RECEIVE_BOOT_COMPLETED`, `POST_NOTIFICATIONS`, `SCHEDULE_EXACT_ALARM`, `WAKE_LOCK`. No camera, no location.

Wear OS

Concern	Stack
UI	Wear Compose
State	Same `:core` repository contract, in-memory backed
Quick capture	**Wear Tile** (`androidx.wear.tiles.TileService`) launches voice composer; **Complication** (`androidx.wear.watchface.complications`) shows a "+" tap target
Reminders	Inherits from `:core` ; `AlarmManager` on the Wear side too

---

5. Quick-capture path latency budget

The 3-second target is the absolute upper bound. The realistic budget is:

Step	Budget
Surface tap to composer focus	< 400 ms
Type / dictate input	user
⏎ → JSON / Room write + UI update	< 200 ms
Reminder schedule (when applicable)	< 100 ms

If any single step exceeds budget, treat it as a regression and reach for a flame graph.

---

6. Observability

`shared/CardCore/Sources/CardCore/Observability/` (Swift) and `android/core/src/main/java/com/americangroupllc/card/core/obs/` (Kotlin) both ship `canImport` -gated stubs:

- `AnalyticsService` — backed by PostHog when present; no-op otherwise.
- `CrashReportingService` — backed by Sentry when present; no-op otherwise.

Both expose a `Surface` enum (`.app` , `.shareExtension` , `.watch` , `.complication` , `.tile`) so the per-surface capture latencies can be distinguished without adding bespoke event names per platform.

See `OBSERVABILITY.md` for the wrapper contract and `SENTRY.md` for the real install steps if you choose to wire the SDKs in.

---

7. CI / CD

Six workflows, each lifted from the Pocket reference scaffold with the four known fixes already baked in (`gradle/actions/setup-gradle@v3`, `import java.util.Properties`, `:core:test` not `testDebugUnitTest`, `androidTestImplementation(composeBom)`).

- **`ci.yml`** — every push: Android unit + lint + assemble debug, iOS/watchOS Swift Package tests, marketing-site lint.
- **`ios.yml`** — iOS app build (sim) + iOS XCUITest + watchOS app build (sim).
- **`android.yml`** — Compose UI smoke on AVD (API 33, x86_64, `-camera-back none`).
- **`marketing.yml`** — deploys `/` to GitHub Pages.
- **`pre-release-tests.yml`** — gate that runs the full matrix before any tag.
- **`release.yml`** — tag-driven Fastlane Match release builds; gracefully no-ops when secrets are absent.

See `RELEASING.md` for the tag → release flow.

OBSERVABILITY

OBSERVABILITY.md

Card — OBSERVABILITY

The contract for analytics + crash reporting wrappers. Both are **off by default** and shipped without any third-party SDK as a hard dependency.

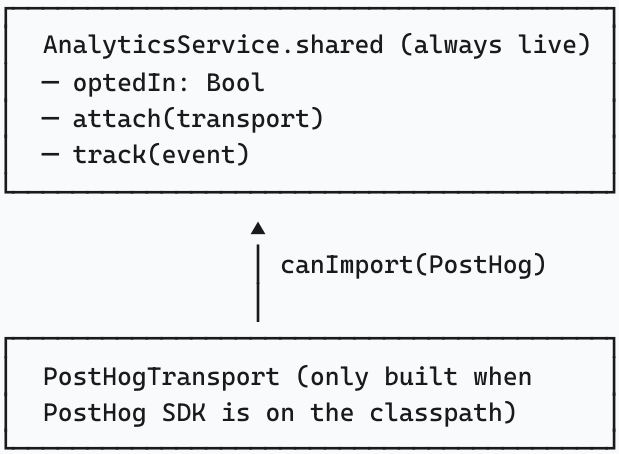
---

1. Why wrappers, not SDKs

Adding a Sentry / PostHog SDK as a hard dependency would:

- Bloat `CardCore` from kilobytes to megabytes.
- Force every consumer of `CardCore` (including the Share Extension) to ship the same SDK, even though the extension shouldn't phone home.
- Break the privacy contract that "no events leave the device unless the user flips a switch in Settings".

Wrapper pattern instead:



If PostHog isn't there, the wrapper exists but `track()` is a no-op. If it is there, you call `AnalyticsService.shared.usePostHog(apiKey:host:)` once at launch, and events flow when `optedIn == true`.

`CrashReportingService` follows the exact same pattern with `Sentry`.

---

2. The `Surface` enum

Every event is tagged with the **capture surface** so per-surface latency can be distinguished. Mirrored across Swift and Kotlin:

Surface	Where it fires
`.app`	Main app composer / action sheet
`.shareExtension`	iOS Share Extension save
`.watch`	watchOS composer (voice or text)
`.complication`	Apple Watch complication tap → composer
`.tile`	Android Quick Settings tile / Wear tile launch

Adding a sixth surface (e.g. a future iOS WidgetKit interactive widget) means: add a case to `Surface` in both `shared/CardCore/Sources/CardCore/Observability/` and `android/core/src/main/java/com/americangroupllc/card/core/obs/`.

---

3. Event taxonomy (v1)

Event	Properties	Fired by
`card_captured`	`surface`, `kind` (note/task/reminder)	every successful save
`card_converted`	`from_kind`, `to_kind`	action sheet
`reminder_scheduled`	`surface`, `delay_minutes`	`ReminderService` schedule call
`reminder_fired`	`surface`	OS notification handler
`card_deleted`	`kind`	swipe-to-delete
`settings_toggled`	`name`, `enabled`	settings screen
`onboarding_completed`	—	last onboarding page

Events are **not** fired when `optedIn == false`. There is also no buffering — opting in mid-session does not flush historical events.

---

4. Crash reports

`CrashReportingService` exposes two methods:

- `capture(error: Error)` — for caught throws / `Result.failure`.
- `capture(message: String)` — for invariant-violation log lines.

Use these sparingly. The point of Sentry on Card is to catch storage corruption (JSON decode failures, Room migration errors) and reminder scheduling failures, **not** to replace `print` debugging.

---

5. Wiring it up

Apple

```
// In AppDelegate.application(_:didFinishLaunchingWithOptions:)

#if canImport(PostHog)
if let key = ProcessInfo.processInfo.environment["POSTHOG_API_KEY"], !key.isEmpty {
    AnalyticsService.shared.usePostHog(
        apiKey: key,
        host: ProcessInfo.processInfo.environment["POSTHOG_HOST"] ??
         "https://us.i.posthog.com"
    )
}
#endif

#if canImport(Sentry)
if let dsn = ProcessInfo.processInfo.environment["SENTRY_DSN"], !dsn.isEmpty {
    CrashReportingService.shared.useSentry(dsn: dsn)
}
#endif

// User-facing toggles drive opt-in:
AnalyticsService.shared.optedIn = settings.analyticsOptedIn
CrashReportingService.shared.optedIn = settings.crashOptedIn
```

Android

```
// In CardApplication.onCreate()

BuildConfig.SENTRY_DSN.takeIf { it.isNotEmpty() }?.let { dsn →
    // when Sentry SDK is present:
    // CrashReportingService.shared.attach(SentryTransport(dsn))
}

BuildConfig.POSTHOG_API_KEY.takeIf { it.isNotEmpty() }?.let { key →
    // PostHog Android setup
    // AnalyticsService.shared.attach(PostHogTransport(key))
}
```

The actual SDK installation steps live in `SENTRY.md` (the same file covers PostHog because the wiring is symmetric).

PRIVACY

PRIVACY.md

Card — PRIVACY

Card collects nothing by default. This file lists every datum the app touches, where it lives, and the language to paste into App Store Privacy Nutrition Labels and the Play Console Data Safety form.

---

1. The one-paragraph version

Card stores every Card you write on the device you wrote it on. It does not have a server, an account system, or a sync mechanism. The only data that ever leaves your device is anonymized crash and product-analytics data — and **only** if you explicitly enable both Settings → "Send crash reports" and "Send anonymous usage data". Both are off by default.

---

2. What each surface touches

Surface	Data	Where it goes
iPhone composer	Card text	Local JSON, App Group container
iOS Share Extension	Selected text from the source app	Same App Group container
iOS reminder	Card id + reminder Date	`UNUserNotificationCenter` (system-only)
Apple Watch composer	Voice recording (transcribed on device) + Card text	Local JSON; **audio is never stored**
Apple Watch complication	Tap-target only	Launches composer; no data of its own
Android composer	Card text	Room (SQLite) in the app's private storage
Android Quick Settings tile	Tap-target only	Launches composer activity; no data of its own
Android reminder	Card id + alarm time	AlarmManager (system-only)
Wear OS composer / tile	Voice recording (transcribed on device) + Card text	Same
Settings	Boolean toggles + theme choice	UserDefaults (Apple) / DataStore (Android)
Sentry / PostHog (when opted in)	Anonymized event names + crash stack traces — **never** Card text	Sentry / PostHog cloud, only after explicit opt-in

Permissions never requested: Camera, Location (precise or coarse), Contacts, Calendar, Photos, Health, Bluetooth, NFC.

---

3. App Store Privacy Nutrition Label (paste into App Store Connect)

Data Type	Linked to user	Tracking	Purpose
Crash data	No	No	App Functionality (opt-in only)
Performance data	No	No	App Functionality (opt-in only)
Diagnostic data	No	No	App Functionality (opt-in only)
Voice recordings	No	No	App Functionality (transcribed locally; never stored or transmitted)

All other categories: **Data Not Collected**.

---

4. Play Console Data Safety (paste into Data Safety form)

Does your app collect or share any of the required user data types?

- Yes — but only **opt-in** crash logs and **opt-in** product analytics.

For each opted-in data type:

- *Crash logs* — Collected, not shared. Encrypted in transit. User can request deletion via Sentry's data-export controls.
- *App diagnostics (analytics)* — Collected, not shared. Encrypted in transit. User can request deletion.

For everything else (location, contacts, files, photos, audio, etc.):

- **Data Not Collected.**

Does your app collect data even when you say it doesn't?

- No.

Is all of the user data collected by your app encrypted in transit?

- Yes (Sentry / PostHog HTTPS endpoints; nothing else leaves the device).

Do you provide a way for users to request that their data be deleted?

- Yes, via Settings → "Erase all data" (wipes Room/JSON locally) and via Sentry / PostHog data-export tooling for opted-in telemetry.

---

5. Voice / Speech specifics

The watch composer and the Quick Settings tile flow accept voice input. The audio:

- Is processed by the **on-device** speech recognizer (`SFSpeechRecognizer` on Apple, `SpeechRecognizer` on Android with the **on-device** flag set).
- Is **never** sent to Apple or Google speech servers (the on-device flag is required at runtime; if the device doesn't support on-device, the voice path is disabled and the composer falls back to typing).
- Is **never** stored — the transcription is the only output and the audio buffer is released immediately.

This is also true for the iOS Share Extension's voice fallback, which is gated behind the same on-device check.

---

6. App Group + Share Extension

The iOS Share Extension writes to `group.com.americangroupllc.card`, the same container the main app reads from. This means:

- The Share Extension can see Cards written by the main app, and vice versa.
- The container is sandboxed by iOS — no other app can read it.
- The container is **not** backed up to iCloud unless the user has iCloud Backup enabled at the OS level (Card itself never opts into iCloud sync).

---

7. Children

Card is rated 4+ on the App Store and "Everyone" on Play. No data is collected that could conflict with COPPA / GDPR-K, because no data is collected by default.

PRODUCTION

PRODUCTION.md

Card — PRODUCTION

What is real, what is fake, and what blocks shipping to the stores.

---

1. What is production-ready

Area	Status
Domain (`Card` , `CardKindTransitions` , `ReminderScheduler` , `CardSorter`)	✅ pure logic, mirrored Swift + Kotlin, fully tested
Storage (`CardStore` JSON, Room)	✅ atomic writes; App Group on Apple
Reminders (UNUserNotifications, AlarmManager)	✅ real OS-scheduled, survives reboot via `BootReceiver`
Quick-capture surfaces	✅ Share Extension, watch complication, Quick Settings tile, Wear tile + complication
Settings (12/24h, theme, opt-ins, erase)	✅ all real and persistent
Observability stubs	✅ canImport/canImport-equivalent gating; SDKs not bundled
6-workflow CI/CD	✅ from day 1, with all 4 known fixes baked in
Marketing site	✅ static + auto-deployed to GitHub Pages

---

2. What is fake / placeholder

Area	Reality
App Icon	Asset-set scaffolding only. Add real PNGs before submitting to either store.
Marketing screenshots	None. Generate from the simulators after the visual smoke pass.
Sentry SDK	Wrapper present, **SDK not added as a dependency**. See `SENTRY.md` to wire up.
PostHog SDK	Same as Sentry.
Privacy nutrition / Privacy Labels	Templates in `PRIVACY.md`; you must paste them into the Play Console + App Store Connect by hand.
App Group entitlement	Declared in `Card.entitlements` and `CardShareExtension.entitlements`, but you must enable the App Group in your Apple Developer account first (Identifiers → App Groups → `group.com.americangroupllc.card`).
Fastlane `Matchfile`	Generated inline by `release.yml` from secrets at run time; no committed Matchfile.
Real Sentry/PostHog DSNs	Read from GitHub Secrets at build time; commit nothing.

---

3. Gap audit before App Store submission

- [ ] **App Icon** — fill `ios/Card/Resources/Assets.xcassets/AppIcon.appiconset/icon-1024.png`.
- [ ] **Launch screen** — current placeholder `LaunchScreen.storyboard` is a plain background. Add a logo if desired.
- [ ] **App Group registration** — enable `group.com.americangroupllc.card` in your developer account; the Share Extension will not write to the shared `CardStore` until this is done.
- [ ] **Notification permission strings** — already in `Info.plist`; review wording in `STORE-PACKAGING.md`.
- [ ] **Speech / Microphone strings** — already in `Info.plist` (for share-ext voice fallback); review wording.
- [ ] **Privacy Nutrition Labels** — paste the table from `PRIVACY.md` §3 into App Store Connect.
- [ ] **Demo account** — App Store reviewers need to test the app; Card is fully on-device, so write a one-liner explaining no account is required.
- [ ] **Watch complication preview screenshots** — required by App Store Connect for any watchOS submission.

---

4. Gap audit before Play Store submission

- [ ] **App Icon** — replace `android/app/src/main/res/mipmap-*` placeholders.
- [ ] **Adaptive icon** — Card requires `mipmap-anydpi-v26/ic_launcher.xml` (foreground/background layers). Not yet shipped.
- [ ] **Play Console Privacy section** — answer the data-safety form using the table in `PRIVACY.md` §4.
- [ ] **Target SDK 34** — already configured in `build.gradle.kts`.
- [ ] `POST_NOTIFICATIONS` **runtime perm** — wired in `MainActivity` on first launch (Android 13+). Test on API 33+ device.
- [ ] `SCHEDULE_EXACT_ALARM` — Play requires a justification statement; see template in `STORE-PACKAGING.md`.
- [ ] **Quick Settings tile screenshot** — required for the tile listing description on Play.
- [ ] **Wear app listing** — separate listing tied to the same package; reuse the AAB and indicate `wear_only`.

---

5. Things to monitor in production (when SDKs are wired)

- **Capture-loop latency** — `surface_tap_to_disk_ms` event from each `Surface` enum case. Alert on p95 > 500 ms.
- **Reminder fire success rate** — per-platform; alert if < 99%.
- **Share-extension write failures** — should be zero; the App Group container is always available.
- **App-launch crash rate** — Sentry; freeze deploys if > 0.5%.

---

6. Things explicitly NOT shipping in v1

These are documented as v1.1 candidates, in priority order:

1. Search across the feed.
2. iOS WidgetKit home-screen widget + Android Glance widget.
3. Today / Upcoming / Done filters when the feed exceeds ~200 cards.
4. Recurring reminders.
5. Tags and saved filters.
6. iCloud / Google Drive optional encrypted backup.
7. AI auto-classification (suggest “this looks like a task”).
8. Card sharing (URL → another Card user → import as a Card).
9. Subscription / IAP scaffolding.

QUICKSTART

QUICKSTART.md

Card — QUICKSTART

The fastest path to a running build on each platform. For deeper architecture context see `DESIGN.md`. For the manual test checklist see `TESTING.md`.

---

0. Common setup

```
git clone git@github.com:AmericanGroupLLC/Card.git
cd Card
```

That's it for cloning. Each platform is self-contained underneath the relevant top-level directory (`ios/`, `watchos/`, `android/`).

---

1. iPhone (macOS, requires Xcode 16)

```
brew install xcodegen
cd ios
xcodegen generate          # produces Card.xcodeproj
open Card.xcodeproj
```

Then in Xcode pick the **Card** scheme + an iPhone 15 simulator and hit ▶.

Or in one shot from the repo root:

```
./scripts/run-ios-sim.sh   # builds + boots iPhone 15 sim + launches Card
```

The Share Extension target (`CardShareExtension`) is generated alongside. To smoke-test sharing, add the simulator to your Mac, open Notes, select some text, tap Share → **Card – Save**.

---

2. Apple Watch (macOS, requires Xcode 16)

```
brew install xcodegen
cd watchos
xcodegen generate          # produces CardWatch.xcodeproj
open CardWatch.xcodeproj
```

Choose the **CardWatch** scheme + an Apple Watch Series 10 (46mm) simulator and hit ▶. To exercise the Quick-capture complication, add it to the watch face from the simulator's complication picker.

---

3. Android phone

```
## requires Android Studio Hedgehog+ (AGP 8.5.0) and JDK 17
cd android
gradle wrapper --gradle-version 8.10   # one-time bootstrap
./gradlew :app:assembleDebug
./gradlew :core:test :app:testDebugUnitTest
```

To run on an emulator:

```
export ANDROID_HOME=~/Library/Android/sdk    # or wherever your SDK lives
export AVD_NAME=Pixel_6_API_34
./scripts/run-android-emulator.sh
```

To exercise the Quick Settings tile: open Quick Settings, hit the pencil (edit) icon, drag the **Card** tile into the active row, then tap it.

---

4. Wear OS

```
cd android
./gradlew :wear:assembleDebug
./gradlew :wear:testDebugUnitTest
```

Run on a Wear emulator:

```
export ANDROID_HOME=~/Library/Android/sdk
export WEAR_AVD_NAME=Wear_Round_API_33
./scripts/run-wear-emulator.sh
```

To exercise the Wear tile: long-press the watch face → "Add tiles" → pick the Card tile.

---

5. Run everything in one shot

```
./scripts/test-all.sh
```

Skips iOS/watchOS on non-macOS hosts and skips Android when Gradle isn't installed — runs whatever is available.

---

6. Release dry-run

Before tagging, sanity-check the binaries that the release workflow would produce:

```
./scripts/bump-version.sh 0.1.0
./scripts/release-dry-run.sh v0.1.0
ls distribution/staging-v0.1.0/
```

See `RELEASING.md` for the full tag → release flow.

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — Card

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets. Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

> **Tagline:** A card for every moment.

Hero — top of marketing site | 4

- viewport: 1280x720
- scroll: top
- wait: 600ms
- caption: {APP_NAME} {VERSION}

Features overview | 5

- viewport: 1280x720
- scroll: 25%
- wait: 600ms
- caption: New in {VERSION}

Highlight reel | 5

- viewport: 1280x720
- scroll: 50%
- wait: 600ms
- caption: Built for everyone

What's next | 5

- viewport: 1280x720
- scroll: 75%
- wait: 600ms
- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try {APP_NAME} today

RELEASING

RELEASING.md

Card — RELEASING

This file describes the tag → release flow. Routine version bumps go through `./scripts/bump-version.sh`; the actual release is tag-driven by GitHub Actions (`release.yml`).

---

1. Pre-release sanity

```
./scripts/test-all.sh                 # every available platform
./scripts/release-dry-run.sh v0.1.0   # build the artefacts that release.yml will build
```

If either fails, fix and re-run. Do not tag with red CI.

The `pre-release-tests.yml` workflow is also runnable manually from the GitHub Actions UI — it is the same gate that `release.yml` uses, so a green manual run predicts a green tag run.

---

2. Bump every manifest

```
./scripts/bump-version.sh 0.1.0
git diff                              # review
git commit -am 'chore(release): v0.1.0'
git push origin main
```

The script edits:

- `android/app/build.gradle.kts` — `versionName = "0.1.0"`
- `android/wear/build.gradle.kts` — `versionName = "0.1.0"`
- `ios/Card/Resources/Info.plist` — `CFBundleShortVersionString = 0.1.0`
- `watchos/CardWatch/Resources/Info.plist` — `CFBundleShortVersionString = 0.1.0`
- `watchos/CardComplication/Info.plist` — `CFBundleShortVersionString = 0.1.0`

It does **not** bump `CFBundleVersion` / `versionCode` — bump those manually when you need a new App Store / Play upload of the same `versionName`.

---

3. Tag and let CI do the rest

```
git tag v0.1.0
git push origin v0.1.0
```

`release.yml` will:

1. Run the full `pre-release-tests.yml` matrix (gate).
2. Build Android phone APK + AAB and Wear APK.
3. Build iOS `.app` (iPhone simulator binary).
4. Build watchOS `.app` (watchOS simulator binary).
5. Zip the marketing site.
6. Create a GitHub Release with all artefacts attached and auto-generated release notes.
7. **If `PLAY_STORE_SERVICE_ACCOUNT_JSON` is set**, upload the AAB to the right Play track (internal / alpha / beta / production based on the tag suffix).
8. **If `APP_STORE_CONNECT_API_KEY_P8_BASE64` is set**, build a signed `.ipa` via Fastlane Match and upload to TestFlight.

When the optional secrets are absent, those jobs gracefully skip with a `::notice::` annotation — the GitHub Release itself still ships.

---

4. Tag suffix → release-channel mapping

Tag form	GitHub Release	Play track	TestFlight
`v0.1.0-internal`	draft + prerelease	`internal`	(manual upload only)
`v0.1.0-alpha.1`	prerelease	`alpha`	TestFlight internal
`v0.1.0-beta.2`	prerelease	`beta`	TestFlight external
`v0.1.0-rc.1`	prerelease	`beta`	TestFlight external
`v0.1.0`	release	`production`	TestFlight → App Store

---

5. Required GitHub Secrets (release.yml)

All optional — `release.yml` no-ops cleanly when missing.

Android

- `ANDROID_KEYSTORE_BASE64` — base64 of `upload.jks`
- `ANDROID_KEYSTORE_PASSWORD`
- `ANDROID_KEY_ALIAS`

- `ANDROID_KEY_PASSWORD`
- `PLAY_STORE_SERVICE_ACCOUNT_JSON` — full Play Console service-account JSON
- `PLAY_STORE_PACKAGE_NAME` — `com.americangroupllc.card`

iOS

- `APP_STORE_CONNECT_API_KEY_ID`
- `APP_STORE_CONNECT_API_ISSUER_ID`
- `APP_STORE_CONNECT_API_KEY_P8_BASE64` — base64 of the `.p8` file
- `APPLE_TEAM_ID`
- `MATCH_PASSWORD` — Fastlane Match repo encryption password
- `MATCH_GIT_URL` — git URL of the Match certificate repo

Both

- `SENTRY_DSN_IOS`, `SENTRY_DSN_ANDROID`, `SENTRY_DSN_WEAR`
- `POSTHOG_API_KEY_IOS`, `POSTHOG_API_KEY_ANDROID`, `POSTHOG_HOST`

When unset, the SDKs no-op; nothing crashes; the binary still ships.

---

6. Hot-fix flow

```
## Branch from the tag
git switch -c hotfix/v0.1.1 v0.1.0

## Cherry-pick or commit the fix
git cherry-pick <fix-commit>

## Bump + tag
./scripts/bump-version.sh 0.1.1
git commit -am 'chore(release): v0.1.1'
git tag v0.1.1
git push origin hotfix/v0.1.1 v0.1.1
```

The same `release.yml` runs.

---

7. Rollback

GitHub Releases can be marked as `prerelease: true` after the fact — that hides the release from the "latest" badge without deleting the artefacts. For Play Store, halt the rollout from the Play Console; for TestFlight, expire the build. **Do not delete tags** — they are the only audit trail of what shipped.

SENTRY

SENTRY.md

Card — SENTRY (and PostHog) install

The wrappers in `shared/CardCore/Sources/CardCore/Observability/` and `android/core/ … /core/obs/` are `canImport`-gated — they no-op when the SDKs aren't on the classpath. This file is the actual install steps for when you decide to wire them up. See `OBSERVABILITY.md` for the contract these SDKs implement.

---

1. iOS / watchOS Sentry

```
## In ios/ (and copy the same to watchos/ if you want the watch covered too)
## Edit ios/project.yml — add Sentry as a SwiftPM dependency

packages:
  CardCore:
    path: ../shared/CardCore
  Sentry:
    url: https://github.com/getsentry/sentry-cocoa.git
    from: 8.30.0
```

Then add `Sentry` as a dependency on the `Card` target:

```
targets:
  Card:
    dependencies:
      - package: CardCore
        product: CardCore
      - package: Sentry
        product: Sentry
```

Re-run `xcodegen generate`. The `#if canImport(Sentry)` blocks in `CrashReportingService.swift` will now compile in.

Set `SENTRY_DSN` as a GitHub Secret; `release.yml` already passes it through to the build environment. The runtime opts in via the user settings toggle.

---

2. iOS / watchOS PostHog

Same shape:

```
packages:
  CardCore:
    path: ../shared/CardCore
  PostHog:
    url: https://github.com/PostHog/posthog-ios.git
    from: 3.10.0
```

Add `PostHog` as a target dependency, re-run `xcodegen generate`.

`shared/CardCore/Sources/CardCore/Observability/AnalyticsService.swift` already has the `#if canImport(PostHog)` extension; nothing else needs to change.

---

3. Android Sentry

```
// android/app/build.gradle.kts
plugins {
    id("io.sentry.android.gradle") version "4.11.0"
}

dependencies {
    implementation("io.sentry:sentry-android:7.13.0")
}
```

Add a `sentry { … }` block:

```
sentry {
    autoUploadProguardMapping.set(false)
    autoInstallation.enabled.set(false)
    includeProguardMapping.set(false)
}
```

Then in `CardApplication.onCreate()`:

```
val dsn = BuildConfig.SENTRY_DSN
if (dsn.isNotBlank()) {
    SentryAndroid.init(this) { options →
        options.dsn = dsn
        options.environment = BuildConfig.BUILD_TYPE
    }
    CrashReportingService.shared.attach(object : CrashTransport {
        override fun capture(throwable: Throwable) {
         Sentry.captureException(throwable) }
        override fun capture(message: String)      { Sentry.captureMessage(message) }
    })
}
```

Then in your `:app:build.gradle.kts`, expose the secret to BuildConfig:

```
android {
    defaultConfig {
        buildConfigField("String", "SENTRY_DSN", "\"${System.getenv("SENTRY_DSN") ?:
         ""}\"")
    }
    buildFeatures { buildConfig = true }
}
```

`release.yml` already plumbs `SENTRY_DSN` (and `SENTRY_DSN_WEAR` for the wear module) through as env vars during the release build.

---

4. Android PostHog

```
dependencies {
    implementation("com.posthog:posthog-android:3.7.0")
}
```

Then in `CardApplication.onCreate()`:

```
val key = BuildConfig.POSTHOG_API_KEY
if (key.isNotBlank()) {
    val cfg = PostHogAndroidConfig(apiKey = key, host = BuildConfig.POSTHOG_HOST)
    PostHogAndroid.setup(this, cfg)
    AnalyticsService.shared.attach(object : AnalyticsTransport {
        override fun track(name: String, properties: Map<String, String>) {
            PostHog.capture(name, properties = properties)
        }
    })
}
```

Mirror the `buildConfigField` lines for `POSTHOG_API_KEY` and `POSTHOG_HOST`.

---

5. Verification

Once the SDKs are wired, run the iOS app + Android app + Wear app on real devices, toggle the opt-ins from Settings, and check:

- Sentry: throw a test exception from a hidden debug menu (or set `SENTRY_DSN` to your dev project, capture a single message at launch).
- PostHog: capture `card_captured` from the composer; verify the event in the PostHog dashboard within 60 seconds.

If neither shows up, the wrapper is no-oping — the most likely cause is `optedIn == false` (the user must flip the Settings toggle).

---

6. Why this isn't on by default

Card's privacy contract says nothing leaves the device unless the user opts in. Shipping the SDKs without a hard opt-in would:

- Force a "we're tracking you" disclosure on first launch.
- Trigger Apple's privacy-nutrition-label "Data Linked to You" requirement even if you're only tracking session counts.
- Make the app feel like every other one in the category.

The wrapper pattern keeps the door open without ever opening it for the user.

SIGNING

SIGNING.md

Signing — Card

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip`; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo:
https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md

---

Android

Secret	Required	Purpose
`ANDROID_KEYSTORE_BASE64`	optional	Base64-encoded `upload.jks`
`ANDROID_KEYSTORE_PASSWORD`	optional	Keystore password
`ANDROID_KEY_ALIAS`	optional	Key alias
`ANDROID_KEY_PASSWORD`	optional	Key password
`PLAY_STORE_SERVICE_ACCOUNT_JSON`	optional	Google Play upload
`PLAY_STORE_PACKAGE_NAME`	optional	Play Console package

A template is at `keystore.properties.example`.

---

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import .p12 to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
`APPLE_CERTIFICATE_BASE64`	optional	.p12 (Distribution)
`APPLE_CERTIFICATE_PASSWORD`	optional	.p12 password
`APPLE_KEYCHAIN_PASSWORD`	optional	Temp keychain pw
`APPLE_PROVISIONING_PROFILE_BASE64`	optional	Wildcard mobileprovision (recommended for CardShareExtension)
`APPLE_TEAM_ID`	optional	10-char team ID
`APPLE_BUNDLE_ID`	optional	Override the default bundle id (`com.americangroupllc.card`)
`APP_STORE_CONNECT_API_KEY_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_ISSUER_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_KEY_P8_BASE64`	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.americangroupllc.card`. Because this app ships with extensions (`CardShareExtension`), use a **wildcard** profile such as `com.americangroupllc.card.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the .ipa is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
- Linux AppImage (unsigned, portable, no install required).
- macOS .dmg (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).

---

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern),

which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

Card — STORE PACKAGING

App Store (iOS + watchOS) and Play Store (Android phone + Wear OS) listing metadata, screenshots, and the data-safety / privacy-label tables.

---

1. App Store Connect — iOS + watchOS

Card ships as **one iOS app with an embedded Apple Watch app**. The `watchos/` Xcode project produces a standalone `.app` for QA convenience, but for store submission the watch app must be embedded inside the iOS `.ipa` as a `watch2-app` extension target.

Migration: standalone watchOS → embedded

In `ios/project.yml`, add the watchOS app and complication as embedded extensions of the iOS target:

```
targets:
  Card:
    type: application
    platform: iOS
    dependencies:
      - target: CardWatchApp        # the watchOS app, brought into ios/project.yml
      - target: CardShareExtension
  CardWatchApp:
    type: watch2-app
    platform: watchOS
    dependencies:
      - target: CardComplication
  CardComplication:
    type: app-extension
    platform: watchOS
```

Then move `watchos/CardWatch/` and `watchos/CardComplication/` source paths into the iOS `project.yml` so they get embedded at archive time.

For now (v0.1.0) the standalone `watchos/CardWatch.xcodeproj` is the development scaffold — submission requires the merge above.

Bundle IDs (already set in `project.yml`)

Target	Bundle ID
Card (iOS)	`com.americangroupllc.card`
CardShareExtension	`com.americangroupllc.card.share`
CardWatchApp	`com.americangroupllc.card.watchkitapp`
CardComplication	`com.americangroupllc.card.complication`

App Group (used by Card + CardShareExtension to share the JSON CardStore): `group.com.americangroupllc.card` — must be created in your developer account under Identifiers → App Groups.

Privacy Nutrition Label (App Store Connect → App Privacy)

Data Type	Linked to user	Tracking	Purpose
Crash data	No	No	App functionality (opt-in only)
Performance data	No	No	App functionality (opt-in only)
Diagnostic data	No	No	App functionality (opt-in only)
Voice recordings (transcribed)	No	No	App functionality (Watch dictation; never stored as audio)

All other categories: **None collected**.

Permission strings (final wording)

- `NSUserNotificationsUsageDescription` — "Card needs to send notifications so the reminders you set actually fire."
- `NSMicrophoneUsageDescription` — "Card – Save can record a quick voice note. Audio is transcribed on device and discarded."
- `NSSpeechRecognitionUsageDescription` — "Card uses on-device speech recognition to turn dictation into Cards. The recordings never leave your phone."

---

2. App Store screenshots

Required sizes (all device classes):

- **iPhone 6.7"** — 1290 × 2796
- **iPhone 6.5"** — 1242 × 2688
- **iPhone 5.5"** — 1242 × 2208 (still required if you target old hardware)

- **iPad Pro 12.9" (3rd gen)** — 2048 × 2732 (only if you ship iPad)
- **Apple Watch (44mm + 49mm Ultra)** — 396 × 484 / 410 × 502

Suggested screen flow (5 screenshots):

1. Hero feed with three cards visible.
2. Composer mid-type ("Buy milk").
3. Action sheet open on a Card with all three convert toggles.
4. Reminder time picker.
5. Watch composer mid-dictation (or watch feed list).

---

3. Play Console — Android phone + Wear OS

Card ships as **one app, two listings**:

- **Phone** — `com.americangroupllc.card`, AAB built from `android/app/`.
- **Wear OS** — same package, separate listing flagged `wearOnly`, AAB built from `android/wear/`.

The Play Console links them automatically when the same `applicationId` is detected.

Data Safety form

Pasteable answers:

- **Does your app collect or share any of the required user data types?**
  - Crash logs / diagnostics — **opt-in only**.
- **Is all of the user data collected by your app encrypted in transit?** — N/A (nothing leaves the device by default).
- **Do you provide a way for users to request that their data be deleted?** — Yes, via Settings → Erase all data (wipes the local Room DB).

`SCHEDULE_EXACT_ALARM` justification

Required since Android 14:

> Card uses `SCHEDULE_EXACT_ALARM` only to fire user-set reminders at the exact minute the user picked. The permission is never used to wake the device for any background task. Reminders are scheduled when the user creates them and re-scheduled by `BootReceiver` after a reboot. There is no server-side trigger.

Quick Settings tile listing

The `<service>` entry in `AndroidManifest.xml` powers the tile listing. Play requires:

- A 24×24 dp white-on-transparent icon (`@drawable/ic_quick_capture_tile`).

- A short label (“Card – Capture”).
- A description (“Tap to add a Card without opening the app.”).

---

4. Play screenshots

- **Phone** — minimum 320 px on the short side, 16:9, max 8 screenshots.
- **Wear** — 384 × 384, max 8 screenshots.
- **Feature graphic** — 1024 × 500 (required for Play listing).

Suggested flow mirrors the iOS list above.

---

5. Asset prep checklist before submission

- [ ] App Icon (iOS 1024×1024 + adaptive Android `mipmap-anydpi-v26/`).
- [ ] Apple Watch complication preview screenshots.
- [ ] Quick Settings tile preview (Android).
- [ ] Wear OS tile preview.
- [ ] Five iPhone 6.7” screenshots.
- [ ] Five Android phone screenshots.
- [ ] Privacy policy URL (host on the marketing site, e.g. `https://americangroupllc.github.io/Card/privacy`).
- [ ] Demo account — Card needs none; explain “no account required” in the reviewer notes field.
- [ ] App Store Promotional text + Description (use the bullets from `index.html`).
- [ ] Play Short description (80 chars max) + Full description.

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
`Card-Setup-X.Y.Z.exe` (NSIS)	no	Windows SmartScreen will prompt on first run; click *More info → Run anyway*.
`Card-X.Y.Z.AppImage`	no	`chmod +x` then double-click on Linux.
`Card-X.Y.Z.dmg`	**no**	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/Card/Card.app"
cp -R "/Volumes/Card/Card.app" /Applications/
```

(or *System Settings → Privacy & Security → Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

Card — TESTING

This file is the manual + automated test matrix. The automated layer is also documented in `DESIGN.md` §7. Use this checklist before tagging a release and to onboard a new contributor.

---

1. Sanity matrix (run on every push)

Layer	Command / surface	Where it runs
Rename completeness	`grep -ri "Pocket|PocketCore|PocketWatch|pocket|pocketwear" . --exclude-dir=.git` returns zero hits	Local
`CardCore` Swift Package	`xcodebuild test -scheme CardCore-Package -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest'`	`ci.yml` + `ios.yml`
Android `:core`	`./gradlew :core:test` (Kotlin/JVM, **not** `testDebugUnitTest`)	`ci.yml`
Android `:app` unit	`./gradlew :app:testDebugUnitTest :app:lintDebug :app:assembleDebug`	`ci.yml`
Wear `:wear` unit	`./gradlew :wear:testDebugUnitTest :wear:assembleDebug`	`ci.yml`
iOS app build (sim)	`xcodegen generate && xcodebuild build … iphonesimulator …`	`ios.yml`
iOS XCUITest smoke	open app → type “Buy milk” → ⏎ → row appears → tap → “Mark as task” → checkbox	`ios.yml`
watchOS build	`xcodegen generate && xcodebuild build … watchsimulator …`	`ios.yml`
Compose UI smoke	`./gradlew :app:connectedDebugAndroidTest`	`android.yml`

---

2. Pure-domain test contract (mirrored case-for-case)

The same scenarios run on both `shared/CardCore/Tests/CardCoreTests/` and `android/core/src/test/java/com/americangroupllc/card/core/`. If you change one, you must change the other or CI will diverge.

`CardKindTransitions`

- `note → task` clears `reminderAt`, leaves `text` / `createdAt` untouched.
- `note → reminder` requires a non-nil reminder Date; reminders set in the past return `nil`.
- `task → note` clears `completedAt`.
- `reminder → task` clears `reminderAt` and adds `completedAt: nil`.
- Identity transition (e.g. `note → note`) is a no-op and returns the same Card.

`ReminderScheduler`

- Next-fire-time math is correct across DST spring-forward and fall-back boundaries.
- Reminders in the past return `nil` (consumer should drop them, not reschedule them).
- Two reminders with the same calendar minute collapse into a single grouped notification with a count badge — verified by `nextFireGrouping` test.

`CardSorter`

- Pinned reminders due in the next 24h sort to the top.
- Completed tasks sort to the bottom regardless of `updatedAt`.
- All other Cards sort by `updatedAt` descending.
- Empty input returns empty output (no crashes, no nil).

---

3. Real-device manual checklist

Run before tagging a release. Mark each ✓ in the PR description.

Capture loop (every device)

- ☐ **Open app** → composer is focused, keyboard is up.
- ☐ Type "Buy milk" → ⏎ → row "Buy milk" appears at the top of the feed in < 200 ms.
- ☐ Tap the row → bottom action sheet shows Mark as task / Set reminder / Done / Edit / Delete.
- ☐ "Mark as task" → row gets a checkbox; tap checkbox → task marks done; row sorts to bottom.
- ☐ Long-press a row → "Edit text" → composer pre-filled; ⏎ → row updates.
- ☐ Swipe left on a row → Delete → row disappears. Reopen app → still gone.

Reminders (iPhone + Android phone)

- ☐ Tap a row → "Set reminder" → pick a time 2 min in the future → save.
- ☐ Background the app. After 2 min, the OS notification fires.
- ☐ Reboot the phone. Set a reminder for the next morning. Reboot → reminder still fires.

iOS Share Extension (iPhone)

- ☐ In Notes (or Safari, or any text source) select a string.
- ☐ Share → **Card – Save**.

- [ ] Relaunch Card → the row is at the top of the feed.
- [ ] Verify: extension never opened the main app; the round-trip was instant.

Apple Watch complication

- [ ] Long-press the watch face → Edit → add the **Card – Quick capture** complication.
- [ ] Tap the complication → composer opens → dictate → save.
- [ ] Pick up the iPhone → row is in the feed (App Group sync).

Android Quick Settings tile

- [ ] Pull down twice → tap pencil → drag the **Card** tile into the active row.
- [ ] Tap the tile → voice composer activity launches.
- [ ] Speak → tap save → row visible in main app feed.

Wear OS tile + complication

- [ ] Long-press watch face → "Add tiles" → Card.
- [ ] Tap tile → composer opens → dictate → save → row visible in Wear feed.
- [ ] Add complication ("+") → tap → composer opens.

Settings (every device)

- [ ] Toggle 12 ↔ 24-hour → reminder times update format.
- [ ] Toggle theme System / Light / Dark → app re-renders without restart.
- [ ] Toggle Sentry opt-in → no crash, even when SDK isn't installed (canImport stub).
- [ ] Toggle PostHog opt-in → no crash, no events sent until SDK is installed.
- [ ] Erase all data → confirm dialog → feed is empty → reopen app → still empty.

---

4. Coverage targets

- `shared/CardCore/Sources/CardCore/Domain/` — **100%** branch coverage. The domain layer is the keystone.
- `shared/CardCore/Sources/CardCore/Storage/` — ≥ 90% line coverage with temp-directory tests.
- iOS view-models — ≥ 60% line coverage; views are exercised by XCUITest.
- `android/core/` — **100%** branch coverage on `domain/` files, mirroring Apple.
- `android/app/` — Compose UI smoke covers the happy path; ≥ 60% line on `feed/`, `composer/`, `settings/` view-models.

Codecov flags (`ios`, `android`) keep these distinguishable on the dashboard; see `codecov.yml`.